

RigPro JSA Platform — Technical Document

Version: 2.0

Last Updated: May 2026

Live Frontend(Only for testing) : <https://jayaminivajira0-ship-it.github.io/JSA/>

Live Backend API(Only for testing): <https://rigpro-backend-production.up.railway.app>

API Docs (Only for testing): <https://rigpro-backend-production.up.railway.app/docs>

Table of Contents

Contents

Table of Contents.....	1
1. Overview	2
2. System Architecture	2
3. Technology Stack.....	3
4. Backend	4
5. Frontend.....	11
6. Business Logic & Workflows.....	13
7. Deployment.....	15
8. Database Schema	17
9. Security	18
10. Testing.....	18
11. Project Structure	19

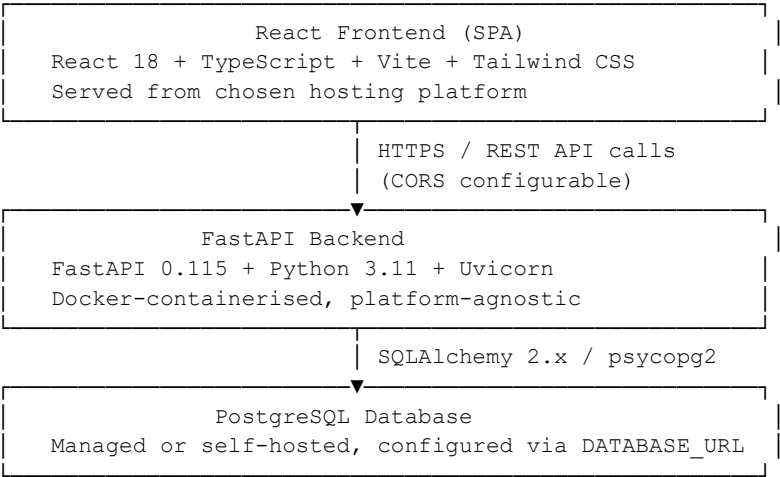
1. Overview

RigPro JSA is a full-stack web application for managing **Job Safety Assessments (JSA)**, inspections, issues, and corrective actions in the marine and construction industries. It is designed for use on vessels, rigs, and worksites where safety compliance and hazard documentation are critical.

Core Capabilities

Module	Description
JSA Management	Create, analyze, submit, and approve Job Safety Assessments with automated hazard detection
Inspections	Conduct template-based inspections with auto-scoring and flagged-item escalation
Issues & Actions	Track safety issues and corrective actions with assignment and status workflows
Notifications	Real-time in-app alerts for assignments, approvals, and critical safety events
Documents	Upload and manage SOPs, procedures, and reference documents
Analytics	KPI dashboards with trend data for inspections, issues, and actions
Assets	Track equipment and machinery with maintenance readings
Contractors	Manage third-party contractor records and compliance documents
Credentials	Track user certifications and license expiry dates
Investigations	Document and manage incident investigations
Heads-Up Alerts	Broadcast safety bulletins to site personnel
Scheduling	Set up recurring inspection schedules with calendar-based occurrences
Offline Sync	Queue JSA submissions when offline and batch-sync on reconnect

2. System Architecture



Key architectural decisions:

- The frontend is a static SPA — it can be served from any static hosting provider (Netlify, Vercel, AWS S3, Nginx, etc.) or bundled into the same container as the backend.
 - The backend is fully containerised (Docker) and platform-agnostic — it runs on any host that supports Docker or Python.
 - The backend auto-creates all database tables on startup using SQLAlchemy metadata and runs lightweight `ALTER TABLE` migrations for column additions.
 - The backend serves its own static file mount (`/storage`) for uploaded PDFs and documents.
 - No message queue or background task runner — notifications and issue creation are synchronous within each request.
-

3. Technology Stack

Why Python and React?

Python (Backend)

Python was chosen for the backend because of its strong ecosystem for data-heavy and safety-critical applications. FastAPI, built on Python, delivers high performance with automatic API documentation (OpenAPI/Swagger) out of the box — which is important for a platform that needs to be auditable and integrable. Python's libraries for PDF generation (ReportLab), data validation (Pydantic), and database ORM (SQLAlchemy) are mature, well-maintained, and widely used in enterprise applications. The language is also readable and maintainable by a broad range of developers, reducing long-term ownership cost.

React (Frontend)

React was chosen for the frontend because it is the most widely adopted UI library, ensuring a large talent pool and long-term support. Its component-based architecture suits a platform like RigPro that has many repeating UI patterns (forms, tables, status badges, modals) across different modules. Combined with TypeScript, it gives compile-time type safety that catches integration bugs between the frontend and the API early. Vite provides near-instant builds and hot reload, making development fast. Tailwind CSS enables consistent, responsive styling without a heavy design framework.

Together, Python + React form a well-understood, low-risk full-stack combination with strong community support, making it easier to hire developers, find documentation, and maintain the platform long-term.

Backend

Package	Version	Purpose
Python	3.11	Runtime
FastAPI	0.115.12	REST API framework
Uvicorn	0.34.2	ASGI server
SQLAlchemy	2.0.40	ORM
Pydantic	2.11.4	Request/response validation
psycopg2-binary	2.9.10	PostgreSQL adapter
ReportLab	4.0.9	PDF generation
Jinja2	3.1.6	Template engine
python-multipart	0.0.20	File upload handling
email-validator	2.2.0	Email format validation

Frontend

Package	Version	Purpose
React	18.3.1	UI framework
TypeScript	5.8.3	Type safety
Vite	5.4.19	Build tool

React Router DOM	6.30.1	Client-side routing
Tailwind CSS	3.4.17	Utility-first styling

Infrastructure

Component	Details
Frontend hosting	Platform TBD — deployable to any static host or served by the backend
Backend hosting	Platform TBD — Docker-containerised, runs on any cloud or on-premise host
Database	PostgreSQL 16 — managed or self-hosted
Container	Docker (python:3.11-slim multi-stage build)
CI/CD	Git-based — specific pipeline to be configured per chosen platform

4. Backend

4.1 Database Models

The backend uses **20 SQLAlchemy ORM models** in `backend/app/models/db_models.py`. All tables are created automatically on startup.

UserDB

Stores all platform users.

Column	Type	Notes
id	String(64) PK	usr-{hex}
email	String(255) unique	Indexed
password_hash	String(255)	Plain text in demo
full_name	String(255)	
role	String(50)	technician, supervisor, admin, view_only
is_active	Boolean	Default true
created_at	DateTime(tz)	Server default now()

TemplateDB

Form schemas for JSAs and inspections.

Column	Type	Notes
id	String(64) PK	
name	String(255)	
category	String(100)	
description	Text	
form_schema	JSON	Sections → questions with score_map, flagged_responses
is_active	Boolean	Default true

created_by	String(64)	
created_at	DateTime(tz)	

JsaRecordDB

Individual Job Safety Assessment records.

Column	Type	Notes
id	String(64) PK	jsa-{hex}
job_number	String(100)	
boat_name	String(255)	
service_log_number	String(100)	
location	String(255)	
date	String(50)	
work_steps	JSON	List of step strings
questionnaire_answers	JSON	Question ID → answer
hazards	JSON	Detected hazards with scores
ppe_list	JSON	Aggregated PPE list
status	String(50)	draft, pending_approval, approved
supervisor_comments	Text	
revision_count	Integer	Default 0
created_by	String(64)	
approved_by	String(64)	
supervisor_signature	Text	Base64 or file URL
pdf_url	String(500)	
summary_json_url	String(500)	
summary_csv_url	String(500)	
created_at	DateTime(tz)	

InspectionRecordDB

Inspection sessions with scoring.

Column	Type	Notes
id	String(64) PK	insp-{hex}
template_id	String(64)	Indexed
template_name	String(255)	
title	String(255)	
site	String(255)	
conducted_by	String(64)	
status	String(50)	in_progress, completed, approved
answers	JSON	Question ID → {value, is_flagged}
flagged_items	JSON	List of flagged question details
score	Float	0-100
total_questions	Integer	

answered_questions	Integer	
started_at	DateTime(tz)	
completed_at	DateTime(tz)	
approved_by	String(64)	
pdf_url	String(500)	

IssueDB

Safety issues, hazards, near-misses, observations.

Column	Type	Notes
id	String(64) PK	iss-{hex}
title	String(255)	
description	Text	
issue_type	String(50)	hazard, near_miss, observation, incident, positive, maintenance
category	String(100)	
site	String(255)	
priority	String(50)	low, medium, high, critical
status	String(50)	open, in_progress, resolved
latitude	Float	Optional geolocation
longitude	Float	Optional geolocation
media_urls	JSON	List of file URLs
custom_answers	JSON	
reported_by	String(64)	
assigned_to	String(64)	
linked_jsa_id	String(64)	Optional
created_at	DateTime(tz)	
updated_at	DateTime(tz)	
resolved_at	DateTime(tz)	

ActionDB

Corrective and preventive actions.

Column	Type	Notes
id	String(64) PK	act-{hex}
title	String(255)	
description	Text	
assigned_to	String(64)	
status	String(50)	to_do, in_progress, done, cant_do
priority	String(50)	low, medium, high
due_date	String(50)	
labels	JSON	List of label strings
action_type	String(50)	corrective, preventive

linked_issue_id	String(64)	Optional
linked_jsa_id	String(64)	Optional
created_by	String(64)	
created_at	DateTime(tz)	
updated_at	DateTime(tz)	

NotificationDB

In-app user notifications.

Column	Type	Notes
id	String(64) PK	ntf-{hex}
user_id	String(64)	Indexed
message	Text	
event_type	String(50)	info, warning, critical, success
link	String(500)	Deep link to resource
is_read	Boolean	Default false
created_at	DateTime(tz)	

Other Models

Model	Purpose
DocumentDB	SOPs and reference documents
TeamDB / TeamMemberDB	Team hierarchy and role assignments
ScheduleDB / ScheduleOccurrenceDB	Recurring inspection schedules
AuditLogDB	Compliance audit trail
AssetDB	Equipment and machinery tracking
ContractorDB	Third-party contractor records
InvestigationDB	Incident investigation reports
HeadsUpDB	Safety bulletins with acknowledgments
CredentialDB	User certifications and license tracking
IssueCommentDB / ActionCommentDB	Threaded comments on issues and actions

4.2 API Endpoints

All endpoints are prefixed with `/api`. The full interactive documentation is available at `/docs`.

Authentication — ``/api/auth``

Method	Path	Description
POST	<code>/login</code>	Authenticate user, returns token and user object

Dashboard — ``/api/dashboard``

Method	Path	Description
GET	<code>/summary</code>	KPIs: JSA counts, completion rate, avg risk score

JSA — `/api/jsa`

Method	Path	Description
GET	``	List all JSA records
POST	/draft	Create a new draft JSA
GET	/{\$id}	Get JSA by ID
POST	/{\$id}/steps	Update work steps
POST	/{\$id}/questionnaire	Update questionnaire answers
POST	/{\$id}/analyze	Run hazard detection and scoring
POST	/{\$id}/submit	Submit for supervisor approval
POST	/{\$id}/approve	Approve JSA with digital signature
GET	/{\$id}/report	Download PDF report
GET	/{\$id}/summary	Export summary JSON/CSV

Inspections — `/api/inspections`

Method	Path	Description
GET	/stats/summary	Inspection KPIs
GET	``	List inspections (filterable by status)
POST	``	Start new inspection from template
GET	/{\$id}	Get inspection details
PATCH	/{\$id}/answers	Auto-save answers during conduct
POST	/{\$id}/complete	Complete inspection, calculate score, auto-create issues
POST	/{\$id}/approve	Supervisor approval
GET	/{\$id}/template	Get template schema
DELETE	/{\$id}	Delete inspection

Issues — `/api/issues`

Method	Path	Description
GET	``	List issues (filterable by status, priority, type)
POST	``	Create issue (auto-notifies based on priority)
GET	/{\$id}	Get issue details
PATCH	/{\$id}/status	Update status and/or assignee
GET	/{\$id}/comments	List comments
POST	/{\$id}/comments	Add comment
DELETE	/{\$id}	Delete issue

Actions — `/api/actions`

Method	Path	Description
GET	``	List actions
POST	``	Create action (notifies assignee)
GET	/{\$id}	Get action

PATCH	/ {id} /status	Update status
GET	/ {id} /comments	List comments
POST	/ {id} /comments	Add comment

Notifications — ``/api/notifications``

Method	Path	Description
GET	``	List user's notifications (latest 50)
POST	``	Create notification
GET	/unread-count	Count of unread notifications
PATCH	/ {id} /read	Mark notification as read
POST	/mark-all-read	Mark all as read
DELETE	/ {id}	Delete notification

Templates — ``/api/templates``

Method	Path	Description
GET	``	List active templates
POST	``	Create template
GET	/ {id}	Get template with schema
PUT	/ {id}	Update template
DELETE	/ {id}	Archive (soft delete)
POST	/ {id} /duplicate	Duplicate template

Analytics — ``/api/analytics``

Method	Path	Description
GET	/analytics/summary	Platform-wide metrics
GET	/analytics/inspections	Inspection trends and score distribution
GET	/analytics/issues	Issues by status, priority, type
GET	/analytics/actions	Action metrics and overdue count
GET	/jsa/analytics	Hazard frequency and average risk

Other Routers

Router	Prefix	Key Operations
Documents	/api/documents	List, upload, version, tag
Assets	/api/assets	CRUD + add readings
Contractors	/api/contractors	CRUD
Credentials	/api/credentials	CRUD (sorted by expiry)
Investigations	/api/investigations	CRUD
Heads-Up	/api/headsup	List, create, acknowledge, deactivate
Schedules	/api/schedules	CRUD + occurrences
Sync	/api/sync	POST <code>/jsa-batch</code> for offline queue upload
Teams	/api/teams	CRUD + member management

Audit	/api/audit	Create and list audit log entries
-------	------------	-----------------------------------

4.3 Core Services

Hazard Detection Engine (`services/hazard\_engine.py`)

The engine maps questionnaire answers and work step text to a library of 25 master hazards.

Detection Methods:

1. **Question Mapping** — Each questionnaire question ID is pre-mapped to a set of hazard IDs. When a question answer triggers the hazard (e.g., answer = "Yes" for a risk question), the associated hazards are activated.
2. **Keyword Matching** — Work step text is scanned for keywords (e.g., lift, crane, electrical, resin, generator) to detect additional hazards not covered by the questionnaire.

Scoring:

- Each hazard has a likelihood (1-3) and severity (1-3)
- Pre-control risk score = likelihood × severity (max 9)
- Controls are assumed to reduce likelihood by 1
- Post-control risk score = (likelihood - 1) × severity
- Scores are embedded in the JSA record

Output:

- Detected hazards with pre/post scores
- Aggregated PPE list (deduplicated across all triggered hazards)
- Control measures per hazard

Example Hazards:

Manual Handling	→ Likelihood 2, Severity 2, PPE: Gloves, Back support
Working at Height	→ Likelihood 3, Severity 3, PPE: Harness, Helmet
Electrical Hazard	→ Likelihood 2, Severity 3, PPE: Insulated gloves, Arc flash PPE
Lifting Equipment	→ Likelihood 3, Severity 3, PPE: Helmet, Gloves, Eye protection

Notification Service (`services/notifications.py`)

```

notify(db, user_id, message, event_type="info", link="")
    # Creates a single NotificationDB record for the given user

notify_supervisors(db, message, event_type="info", link="")
    # Queries all users with role in ["supervisor", "admin"]
    # Calls notify() for each

```

PDF Service (`services/pdf\_service.py`)

Generates JSA PDF reports using ReportLab:

- Job header: boat name, location, date, job number
- Work steps table
- Hazards table with pre/post risk scores
- PPE and control measures
- Supervisor signature (embedded as base64 image)
- Brand colours: #377133 (dark green), #499241, #c2e5b2 (light green)

Seed Service (`services/seed.py`)

Runs once on startup to create demo data if the database is empty:

- 3 default users (admin, technician, supervisor)
- Marine Vessel Safety Inspection template
- Medical Clearance template

ID Generation (`services/store.py`)

```
new_id(prefix: str) -> str
# Returns f"{prefix}-{secrets.token_hex(4)}"
# Examples: "jsa-a3f1c2d4", "insp-b8e2a1f3", "ntf-d4c9a7b2"
```

4.4 Authentication

> **Note:** The current implementation uses simplified token-based auth suitable for a demo environment. See [Security](#9-security) for production recommendations.

Login flow:

3. Client sends POST /api/auth/login with { email, password }
4. Backend queries UserDB by email, compares password directly
5. Returns { token: "demo-token-{user_id}", user: { id, email, full_name, role } }
6. Frontend stores token in localStorage as rigpro_token
7. All subsequent requests include Authorization: Bearer {token} header

Default demo credentials:

Email	Password	Role
admin@rigpro.local	admin123	admin
tech@rigpro.local	tech123	technician
supervisor@rigpro.local	super123	supervisor

5. Frontend

5.1 Pages & Routes

The frontend uses React Router v6 with `BrowserRouter`. The `basename` is configurable via the `VITE_BASE_URL` build variable to match the deployment sub-path on the chosen hosting platform.

Route	Page Component	Access
/	LoginPage	Public
/dashboard	DashboardPage	All roles
/inspections	InspectionsPage	All roles
/inspections/conduct/:id	InspectionConductPage	All roles
/inspections/report/:id	InspectionReportPage	All roles
/jsa/review/:id	ReviewPage	All roles
/jsa/report/:id	JsaReportPage	All roles
/supervisor	SupervisorPage	Supervisor, Admin
/documents	DocumentsPage	All roles
/sync	SyncPage	All roles
/issues	IssuesPage	All roles
/actions	ActionsPage	All roles

/templates	TemplatesPage	All roles
/templates/new	TemplateBuilderPage	All roles
/templates/edit/:id	TemplateBuilderPage	All roles
/assets	AssetsPage	All roles
/analytics	AnalyticsPage	All roles
/contractors	ContractorsPage	All roles
/credentials	CredentialsPage	All roles
/headsup	HeadsUpPage	All roles
/investigations	InvestigationsPage	All roles
/scheduling	SchedulingPage	All roles

5.2 Components

Layout (`components/Layout.tsx`)

The shared layout wrapper used by all authenticated pages. Provides:

- **Sidebar navigation** with role-based menu visibility (Supervisor Queue hidden for technicians)
- **Collapsible sidebar** — open/closed state persisted in `localStorage`
- **Header** with page title and notification bell
- **NotificationBell** — polls every 30 seconds for unread count; dropdown shows latest notifications colour-coded by type; supports mark-read and click-to-navigate
- **Logout button**

Notification colour coding:

Event Type	Colour
critical	Red dot
warning	Amber dot
success	Green dot
info	Blue dot

5.3 API Client

Located in `frontend/src/api.ts`.

```
const API_BASE = import.meta.env.VITE_API_URL ?? "";

async function api<T>(path: string, options?: RequestInit): Promise<T>
```

- Reads `rigpro_token` from `localStorage` on every call
- Adds `Content-Type: application/json` and `Authorization: Bearer {token}` headers
- Throws on non-OK responses with the response body as the error message
- Returns parsed JSON typed as `T`

Typed exports also available:

- `notificationsApi.list(userId)`
- `notificationsApi.unreadCount(userId)`
- `notificationsApi.markRead(id)`
- `notificationsApi.markAllRead(userId)`

5.4 State Management

The app uses React's built-in hooks with no external state library:

- `useState / useEffect` for component-local data
- `localStorage` for token and sidebar state persistence
- No global store — each page fetches its own data on mount

5.5 TypeScript Types (`frontend/src/types.ts`)

```
type User = { id: string; email: string; full_name: string; role: "technician" | "supervisor" | "admin" | "view_only" }
type JsaRecord = { id, job_number, boat_name, status, hazards, ppe_list, ... }
type Hazard = { id, name, controls, pre_score, post_score, likelihood, severity, ppe }
type Issue = { id, title, issue_type, priority, status, site, reported_by, ... }
type Action = { id, title, assigned_to, status, priority, due_date, labels, ... }
type FormTemplate = { id, name, form_schema: { sections: Section[] } }
type Notification = { id, message, event_type, link, is_read, created_at }
```

6. Business Logic & Workflows

6.1 JSA Workflow

1. DRAFT
Technician creates JSA with:
 - Job number, boat name, location, date
 - List of work steps (free text)
2. QUESTIONNAIRE
Technician answers 25 yes/no safety questions covering categories: manual handling, tools, lifting, electrical, height, confined space, etc.
3. ANALYZE
 POST /api/jsa/{id}/analyze
 → Hazard Engine detects hazards from:
 - a) Questionnaire answer mapping
 - b) Keyword scan of work steps
 → Assigns pre/post risk scores
 → Generates PPE list
 → Saves hazards + ppe_list to JsaRecordDB
4. SUBMIT
 POST /api/jsa/{id}/submit
 → status = "pending_approval"
 → Supervisors notified
5. REVIEW & APPROVE
 Supervisor opens JSA, reviews hazards and steps
 POST /api/jsa/{id}/approve
 → Saves supervisor_comments + digital signature
 → status = "approved"
 → PDF report generated
6. DOWNLOAD
 GET /api/jsa/{id}/report → PDF
 GET /api/jsa/{id}/summary → JSON/CSV export

6.2 Inspection Workflow

1. START
POST /api/inspections

- Select template, enter site and title
- Creates InspectionRecordDB with status = "in_progress"

2. CONDUCT

- PATCH /api/inspections/{id}/answers
- Auto-saves after each answer
- Answer structure: { question_id: { value, is_flagged } }

3. COMPLETE

- POST /api/inspections/{id}/complete
- Calculates score using _calc_score()
- For each non-skipped flagged item:
 - Creates IssueDB (type=hazard, priority=high, status=open)
- Notification logic:
 - Score < 70% → notify_supervisors (CRITICAL)
 - Flags exist → notify_supervisors (WARNING)
 - Clean result → notify_supervisors (INFO)
- status = "completed"

4. APPROVE

- POST /api/inspections/{id}/approve
- status = "approved"
- approved_by set to supervisor user ID

6.3 Issue & Action Management

Issue status lifecycle:

open → in_progress → resolved

Notification triggers on issues:

- Created with priority = "high" or "critical" → notify_supervisors(..., event_type="critical")
- Created with priority = "medium" → notify_supervisors(..., event_type="warning")
- assigned_to set → notify(db, assigned_to, ..., event_type="info")
- Status changed to resolved → notify(db, reported_by, ..., event_type="success")

Action status lifecycle:

to_do → in_progress → done
→ cant_do

Notification triggers on actions:

- Created with assigned_to set → notify(db, assigned_to, ..., event_type="info")
- Status changed to complete → notify(db, created_by, ..., event_type="success")

6.4 Notification System

All notifications are created synchronously within the request that triggers them. There is no background queue.

```
# Single user notification
notify(db, user_id="usr-abcl23", message="Your issue has been resolved",
       event_type="success", link="/issues/iss-xyz")

# Broadcast to all supervisors and admins
notify_supervisors(db, message="HIGH priority issue reported: Fuel leak",
                  event_type="critical", link="/issues/iss-xyz")
```

Frontend polling:

The NotificationBell component calls GET /api/notifications/unread-count every 30 seconds and updates the badge count accordingly.

6.5 Inspection Scoring

The `_calc_score(form_schema, answers)` function in `backend/app/routers/inspections.py`:

```
def _calc_score(form_schema: dict, answers: dict) -> float:
    earned = 0.0
    max_pts = 0.0
    for section in form_schema.get("sections", []):
        for question in section.get("questions", []):
            if question.get("type") != "multiple_choice":
                continue
            score_map = question.get("score_map", {})
            valid_values = [v for v in score_map.values()
                           if isinstance(v, (int, float))]
            if not valid_values:
                continue
            q_max = max(valid_values)
            max_pts += q_max
            answer = answers.get(question["id"], {})
            value = answer.get("value") if isinstance(answer, dict) else None
            if value is not None:
                pts = score_map.get(str(value))
                if isinstance(pts, (int, float)):
                    earned += pts
    if max_pts == 0:
        return 100.0          # No scoreable questions = pass
    return round((earned / max_pts) * 100, 1)
```

Rules:

- Only `multiple_choice` questions with a `score_map` contribute to the score
- N/A options with `null` in `score_map` are excluded from the max
- Unanswered questions count as 0 points earned
- Templates with no scored questions return 100%

6.6 Hazard Detection Engine

The 25 master hazards each contain:

- `id` — integer identifier
- `name` — hazard name
- `likelihood / severity` — base risk values (1-3)
- `controls` — list of control measures
- `ppe` — list of required PPE items
- `keywords` — work step keywords that trigger detection
- `question_ids` — questionnaire question IDs that trigger detection

Score Calculation:

```
Pre-control score = likelihood × severity
Post-control score = max(1, likelihood - 1) × severity
Risk level: 1-3 = Low, 4-6 = Medium, 7-9 = High
```

7. Deployment

7.1 Deployment Architecture

The platform is fully containerised and platform-agnostic. The deployment target has not yet been finalised. The application is ready to be deployed to any of the following environments:

Option	Frontend	Backend	Database
Cloud PaaS	Vercel / Netlify / Azure Static Web Apps	Azure App Service / AWS ECS / GCP Cloud Run	Azure Database for PostgreSQL / AWS RDS / Cloud SQL
Container platform	Served by backend or CDN	Docker container	Managed PostgreSQL
On-premise / VPS	Nginx static files	Docker + Nginx reverse proxy	Self-hosted PostgreSQL
Full Docker Compose	Nginx container	FastAPI container	PostgreSQL container

7.2 Container Build

The backend is packaged as a Docker image using `python:3.11-slim`.

Backend Dockerfile (backend/Dockerfile):

```
FROM python:3.11-slim
WORKDIR /app
RUN apt-get update && apt-get install -y --no-install-recommends \
    libpango-1.0-0 libpangoft2-1.0-0 libgdk-pixbuf-xlib-2.0-0 \
    libffi-dev shared-mime-info \
    && rm -rf /var/lib/apt/lists/*
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY app ./app
EXPOSE 8000
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

A **multi-stage Dockerfile** at the repository root also exists for single-container deployments that bundle the React build together with the backend, serving the frontend as static files through FastAPI.

7.3 Environment Variables

Backend — required at runtime:

Variable	Example Value	Purpose
DATABASE_URL	postgresql://user:password@host:5432/dbname	PostgreSQL connection string
PORT	8000	Port for Uvicorn to listen on

Frontend — required at build time:

Variable	Example Value	Purpose
VITE_API_URL	https://api.yourdomain.com	Backend base URL (empty if same-origin)
VITE_BASE_URL	/	Asset base path (set to sub-path if not served from root)

> If the frontend and backend are served from the same domain (e.g., bundled in one container), `VITE_API_URL` should be left empty — all API calls will be relative to the same origin.

7.4 Local Development

With Docker Compose (recommended):

```
# Starts PostgreSQL + FastAPI backend together
docker compose up --build
```

```
# Backend API:  http://localhost:8000
# API Docs:    http://localhost:8000/docs
```

Without Docker:

```
# Terminal 1 - Backend
cd backend
pip install -r requirements.txt
uvicorn app.main:app --reload --port 8000

# Terminal 2 - Frontend
cd frontend
npm install
npm run dev
# Opens at http://localhost:5173
# Dev server proxies /api/* → http://localhost:8000
```

Database: When `DATABASE_URL` is not set, the backend automatically falls back to a local SQLite file (`rigpro.db`) — no PostgreSQL installation required for local development.

8. Database Schema

UserDB

- └─ TeamMemberDB (user_id FK)
- └─ JsaRecordDB (created_by FK)
- └─ InspectionRecordDB (conducted_by FK)
- └─ IssueDB (reported_by / assigned_to FK)
- └─ ActionDB (assigned_to / created_by FK)
- └─ NotificationDB (user_id FK)
- └─ CredentialDB (user_id FK)
- └─ AuditLogDB (user_id FK)

TemplateDB

- └─ JsaRecordDB (template-based JSAs)
- └─ InspectionRecordDB (template_id FK)
- └─ ScheduleDB (template_id FK)

ScheduleDB

- └─ ScheduleOccurrenceDB (schedule_id FK)

IssueDB

- └─ IssueCommentDB (issue_id FK)
- └─ ActionDB (linked_issue_id FK)
- └─ InvestigationDB (linked_issue_id FK)

ActionDB

- └─ ActionCommentDB (action_id FK)

InspectionRecordDB

- └─ IssueDB (auto-created from flagged items)
- └─ InvestigationDB (linked_inspection_id FK)

TeamDB

- └─ TeamMemberDB (team_id FK)

DocumentDB (standalone)

AssetDB (standalone)

ContractorDB (standalone)

9. Security

Current Implementation (Demo-Level)

Area	Current State
Passwords	Plain-text comparison — no hashing
Tokens	demo-token-{user_id} — no expiry, no JWT signature
CORS	allow_origins=["*"] — all origins allowed
Input validation	Pydantic schemas validate all request bodies
SQL injection	Protected by SQLAlchemy parameterised queries
File uploads	No file type or size validation
Audit trail	AuditLogDB table exists but not fully wired to all mutations

Recommended for Production

Area	Recommendation
Passwords	bcrypt with passlib[bcrypt]
Tokens	JWT with expiry (15 min access, 7 day refresh) using python-jose
CORS	Restrict to known frontend domain only
HTTPS	Ensure TLS termination is configured at the hosting layer — no plain HTTP in production
File uploads	Validate MIME type and enforce max size (e.g., 10 MB)
Rate limiting	Add slowapi middleware on /api/auth/login
Secrets	Store all credentials in the hosting platform's secret/environment variable store, never in code

10. Testing

Tests are located in `backend/tests/` and use `pytest` with an in-memory SQLite database.

Test Files

File	Tests	Coverage
<code>test_score_calc.py</code>	10	<code>_calc_score()</code> formula: perfect, zero, partial, weighted, unanswered, no-schema, empty, N/A handling, threshold checks
<code>test_notifications.py</code>	12	<code>notify()</code> , <code>notify_supervisors()</code> , issue priority triggers, action assignment triggers, inspection flagging and skipping

Running Tests

```
cd backend
pip install pytest pytest-mock httpx
pytest tests/ -v
```

All 22 tests pass.

Key Test Scenarios

```
# Score formula
test_perfect_score_all_yes()           # → 100.0
test_zero_score_all_no()               # → 0.0
test_partial_score_mixed_answers()     # → 50.0
test_unequal_weights()                 # → 66.7
test_na_option_with_null_score()       # → 100.0 (N/A excluded from max)
test_score_below_70_threshold()        # → 30.0 (triggers CRITICAL alert)

# Notifications
test_notify_creates_notification()
test_notify_supervisors_sends_to_supervisor_and_admin()
test_issue_high_priority_notifies_supervisors() # event_type = "critical"
test_issue_medium_priority_notifies_as_warning() # event_type = "warning"
test_issue_resolution_notifies_reporter()       # event_type = "success"
test_action_creation_notifies_assignee()
test_action_completion_notifies_creator()       # event_type = "success"
test_inspection_auto_creates_issues_for_flags() # creates IssueDB
test_inspection_skipped_flags_do_not_create_issues()
```

11. Project Structure

```
JAS/
├── Dockerfile                               # Multi-stage: Node 20 (React) + Python 3.11 (FastAPI)
├── docker-compose.yml                       # PostgreSQL 16 + FastAPI, local dev
├── railway.toml                             # Deployment config (platform-specific, to be updated)
├──
├── backend/
│   ├── Dockerfile                           # Backend container image
│   ├── requirements.txt                     # Python dependencies
│   └── app/
│       ├── main.py                         # FastAPI app init, router registration, lifespan
│       ├── core/
│       │   └── db.py                       # SQLAlchemy engine, session factory, DB URL handling
│       ├── models/
│       │   └── db_models.py                # 20 SQLAlchemy ORM models
│       ├── schemas/
│       │   └── models.py                   # Pydantic request/response models
│       ├── routers/
│       │   ├── auth.py
│       │   ├── jsa.py
│       │   ├── inspections.py              # _calc_score(), complete_inspection()
│       │   ├── issues.py                  # Priority-based notifications
│       │   ├── actions.py                 # Assignment notifications
│       │   ├── notifications.py           # CRUD endpoints
│       │   ├── templates.py
│       │   ├── documents.py
│       │   ├── analytics.py
│       │   └── assets.py
```

```

├── contractors.py
├── credentials.py
├── headsup.py
├── investigations.py
├── schedules.py
├── teams.py
├── sync.py
├── audit.py
├── dashboard.py
├── forms.py
├── services/
│   ├── notifications.py      # notify() and notify_supervisors()
│   ├── hazard_engine.py     # 25 hazard library + detection
│   ├── pdf_service.py       # ReportLab PDF generation
│   ├── rbac.py              # Role-based access checks
│   ├── seed.py              # Demo data seeding
│   ├── store.py             # ID generation
│   └── mappers.py           # DB model → Pydantic schema converters
├── backend/tests/
│   ├── test_score_calc.py    # 10 scoring tests
│   └── test_notifications.py # 12 notification trigger tests
├── frontend/
│   ├── package.json
│   ├── vite.config.ts        # base: "/JSA/", dev proxy → localhost:8000
│   ├── tailwind.config.js    # brand-* colour palette (brand-50 → brand-900)
│   ├── .env.production       # VITE_API_URL, VITE_BASE_URL
│   ├── netlify.toml          # Alternative: Netlify proxy config
│   └── src/
│       ├── main.tsx          # React entry, BrowserRouter with basename
│       ├── App.tsx           # Route definitions, auth state
│       ├── api.ts            # Typed API client helper
│       ├── types.ts          # TypeScript interfaces
│       ├── offlineQueue.ts   # localStorage-based offline queue
│       ├── vite-env.d.ts     # Vite import.meta.env types
│       ├── styles.css        # Global styles + Tailwind directives
│       ├── components/
│       │   └── Layout.tsx    # Sidebar, header, NotificationBell
│       └── pages/
│           ├── LoginPage.tsx
│           ├── DashboardPage.tsx
│           ├── InspectionsPage.tsx
│           ├── InspectionConductPage.tsx
│           ├── InspectionReportPage.tsx
│           ├── IssuesPage.tsx
│           ├── ActionsPage.tsx
│           ├── TemplatesPage.tsx
│           ├── TemplateBuilderPage.tsx
│           ├── AssetsPage.tsx
│           ├── AnalyticsPage.tsx
│           ├── ContractorsPage.tsx
│           ├── CredentialsPage.tsx
│           ├── HeadsUpPage.tsx
│           ├── InvestigationsPage.tsx
│           ├── SchedulingPage.tsx
│           ├── DocumentsPage.tsx
│           ├── SupervisorPage.tsx
│           ├── JsaWizardPage.tsx
│           ├── ReviewPage.tsx
│           ├── JsaReportPage.tsx
│           └── SyncPage.tsx

```